```python
from users.learner import Learner
from users.user import User
from data import *


class Parent(User):
    """
    This is the class definition for the Parent class.
    """

    def __init__(self, email, first_name, last_name, username, password,
    date_of_birth):
        """
        Constructor method for the Parent class.

        :param email: The email address of the parent.
        :param first_name: The first name of the parent.
        :param last_name: The last name of the parent.
        :param username: The username of the parent.
        :param password: The password of the parent.
        :param date_of_birth: The date of birth of the parent.
        """
        super().__init__(
            email, first_name, last_name, username, password, role="Parent"
        )
        self.date_of_birth = date_of_birth
        self.children = []  # list to store associated learners/children

    def add_child(self):
        """
        Associate a learner/child with this parent by matching a name from users.txt.
        """
        # Load users from users.txt
        with open("./data/users.txt", "r") as file:
            users = file.readlines()

        learners = [
            user.strip().split(",")
            for user in users
            if user.strip().split(",")[-1] == "Learner"
        ]
        # Let the parent input the name of their child
        first_name_input = input("\nEnter child's first name: ").strip()
        last_name_input = input("Enter child's last name: ").strip()

        # Search for a matching learner
        matching_learner = None
        for learner in learners:
            if (
                learner[0].lower() == first_name_input.lower()
                and learner[1].lower() == last_name_input.lower()
            ):
                matching_learner = learner
                break

        if matching_learner:
            email = matching_learner[-1]
            first_name = matching_learner[0]
            last_name = matching_learner[1]
            dob = matching_learner[2]
            username = matching_learner[3]
            password = matching_learner[4]

            child = Learner(email, first_name, last_name, username, password, dob)

            if child not in self.children:
                self.children.append(child)
                print(f"{first_name} {last_name} has been added as your child.")
            else:
                print(f"{first_name} {last_name} is already associated with you.")
        else:
            print(
                f"No learner found with the name {first_name_input}
```

```
                    {last_name_input}."
                )

    def provide_feedback(self):
        """
        Provide feedback to a specific educator.
        """
        # Load users from users.txt
        with open("./data/users.txt", "r") as file:
            users = file.readlines()

        educators = [
            user.strip().split(",")
            for user in users
            if user.strip().split(",")[-1] == "Educator"
        ]

        if not educators:
            print("No educators available.")
            return

        # Display educators
        print("\nAvailable educators:")
        for idx, educator in enumerate(educators, 1):
            print(f"{idx}. {educator[0]} {educator[1]}")

        # Let the parent select an educator by index
        choice = int(input("\nSelect an educator to provide feedback to: "))

        if 1 <= choice <= len(educators):
            selected_educator = educators[choice - 1]
            feedback = input(
                f"\nProvide feedback for {selected_educator[0]}
                {selected_educator[1]}: "
            )

            # As before, we're just printing the feedback for demonstration purposes.
            # In a complete implementation, you'd associate this feedback with the
            selected educator.
            print(
                f"\nProvided feedback to {selected_educator[0]}
                {selected_educator[1]}: {feedback}"
            )
        else:
            print("Invalid choice.")

    def dashboard(self):
        while True:
            print(f"\nWelcome {self.first_name}\n")
            print("1. Add Child")
            print("2. See Children Progress")
            print("3. Provide Feedback")
            print("4. Edit profile")
            print("5. Logout")
            choice = input("Enter your choice: ")

            if choice == "1":
                self.add_child()
            elif choice == "2":
                if not self.children:
                    print("No children associated yet.")
                    continue
                print("\nSelect a child:")
                for idx, child in enumerate(self.children, 1):
                    print(f"{idx}. {child.get_first_name()} {child.get_last_name()}")

                child_choice = int(input("Enter your choice: "))
                if 1 <= child_choice <= len(self.children):
                    selected_child = self.children[child_choice - 1]
                    progress = self.see_progress(selected_child)
                    print(progress)
                else:
                    print("Invalid choice.")
```

```
142                 elif choice == "3":
143                     self.provide_feedback()
144                 elif choice == "4":
145                     self.edit_profile()
146                 elif choice == "5":
147                     break
148                 else:
149                     print("Please choose a valid option.")
150 
```